Microprocessor Systems Project
Final Project Report

Ahnaf Zareef – C01 – zareefa – 400567499

Professor: Dr. Yaser Haddara, Dr. Shahrukh Athar,
Dr. Mohammed Elamien

McMaster University

COMPENG 2DX3

April 7th, 2026

As a future member of the engineering profession, the student is responsible for performing the required work in an honest manner, without plagiarism and cheating. Submitting this work with my name and student number is a statement and understanding that this work is our own and adheres to the Academic Integrity Policy of McMaster University and the Code of Conduct of the Professional Engineers of Ontario.

AZ-2DX3-DELVIE-01 Spatial Mapper™

1. Device Overview

1.1. Features

- **36 MHz bus speed** for scanning, motor control and sensor polling.
- **Up to 4000 mm ranging** using the VL53L1X.
- **Single-button scan control** using PJ0 as an interrupt driven start/stop.
- **Operating Voltage** using USB 3.3 V to MSP432E401Y MCU, and 2.6V – 5.5V for VL53L1X.
- **Receiver MATLAB** code for XYZ conversion, file exporting and plotting
- **$200 Implementation Cost**
- **1024 KB Flash & 256 KB SRAM** on MSP432E401Y MCU.
- **Serial Communication** using UART0 at 115200 bps.
- **Status LEDs** for measurement activity and scan completion
- **Multi-slice spatial capture.**
- **128-angle scan mode** spanning 0º to 360º.

1.2 General Description

This report defines the entire design, implementation and testing process of the Delvie Spatial Mapper, a spatial mapping device built to offer a better alternative to expensive LIDAR mapping technologies for small scale integration. Utilizing the TI-MSP432E401Y microcontroller, Pololu VL53L1X Time-of-Flight sensor and a stepper motor, the device scans vertical locations into measurement data and sends the recorded data back to the PC for real-time reconstruction of the scanned region into a 3D map. It is important to note that the Time-of-Flight sensor already performs ADC conversion along with optical ranging, and as a result, does not have to be handled internally using the MCU by implementing an ADC stage [1, 4].

Starting when PJ0 is pressed, the MCU rotates the sensor through predefined angles, waits until an interrupt is fired to indicate the completion of one scan, reads the measurement data and transmits the results serially through UART to the PC as comma delimited text for acquisition and mapping. If PJ0 is pressed during run-time, the interrupt handler sends a stop request and the scan is terminated. Thus, it is imperative that the button has both a start and stop function depending on the state.

On the retrieval side, MATLAB receives each scan, filters through invalid distance values, converts into cartesian coordinate and saves the results into a .xyz file, to then plot the final 3D map. It is important to note that the convention for coordinate position follows suit with what is defined within course test, of which the y-z plane is one vertical scan and the x-axis represents fixed displacement between slices

since the VL53L1X is two-dimensionally driven [1]. The system is collecting data in a form that can be reconstructed into something visible and useful.

1.3 Functional Block Diagram

Figure 1 shows the functional block diagram. and Figure 2 shows the data path.

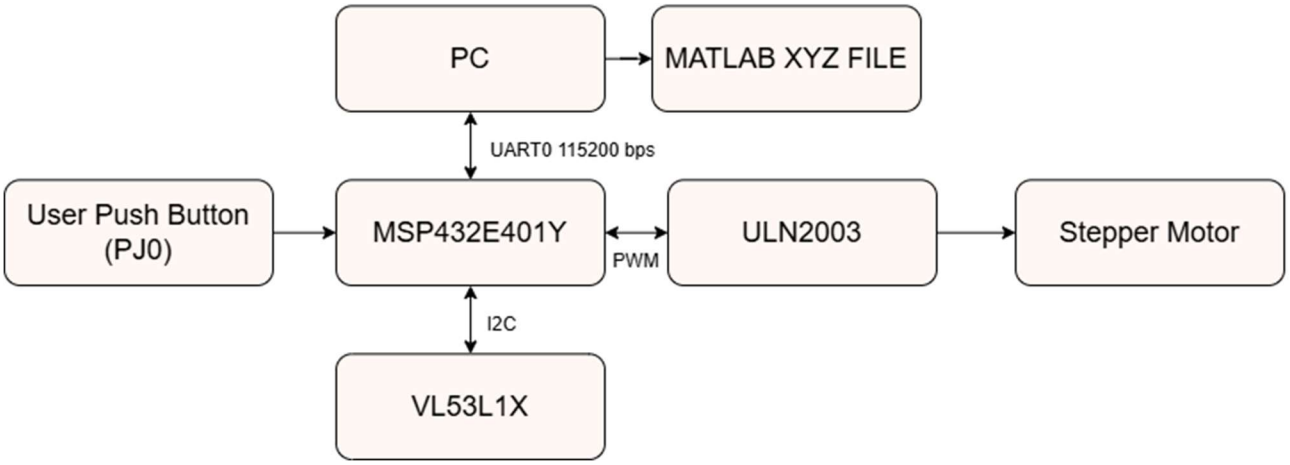

Figure 1: Functional Block Diagram and Data Path

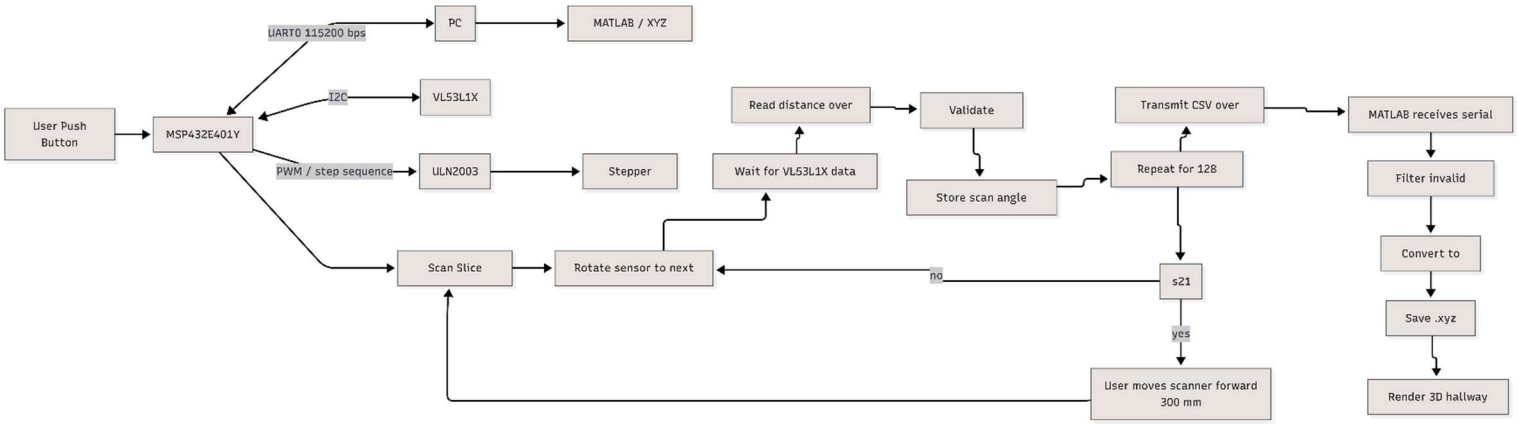

Figure 2: Data Path

2. Device Characteristics Table

Feature	Description
Microcontroller	TI-MSP432E401Y SimpleLink™ 32-bit Arm Cortex-M4F MCU with ethernet, CAN, 1MB Flash and 256kB RAM [2]
Bus Speed	36 MHz
MCU Memory	1024 KB Flash, 256 KB SRAM [2]
TOF Sensor	Pololu VL53L1X ToF sensor, 940 nm emitter, SPAD receiver, long-range mode to 4 m [4]
Sensor Interface and Protocol	I2C0 on PB2 (SCL) and PB3 (SDA), device address of 0x29
UART Interface and Protocol	UART0 on PA0 and PA1 with 115200 bps over virtual COM.
User Input	PJ0 interrupt-driven start/stop button
Status LEDs	PN1: Measurement LED, PN0: UART LED, PF4: Completed Scan LED.
Stepper Ports	PH0-PH3 pins
Receiver Software	MATLAB Serial Data Acquisition and 3D Mapping
Displacement Definition in Code	Demo: 100mm between 3 Scans, hallway: 300 mm between scans.
Operating Voltage	USB-Power to MCU: 3.3V. Sensor: 2.6V-5.5V in VIN pin [5].

3. Detailed Description

3.1 Distance Measurement

Acquisition

This section focuses on the backend of how distance measurement is acquired and how the measurement eventually becomes the building block of the final 3D model.

The VL53L1X is a laser-ranging TOF sensor, integrating a 940 nm infrared emitter and a single-photon avalanche diode (SPAD) receiving array [4]. Conventionally, an analog voltage reading is converted externally to determine distance. With the VL53L1X, the device measures the optical pulse and returns an absolute distance value. Simply put, this means that sensor itself handles the acquisition, processing and conversion.

At the physical level, the distance is calculated based on the round-trip travel time of light:

$$d = \frac{ct}{2}$$

where d is the one-way distance to the perpendicular target surface, c is the speed of light and t is the measured time required for the emitted light to travel to and from the surface. Consider the example

where it takes 10 ns for this ray of light to travel. In this case, the distance can be calculated to be:

$$d = \frac{(3.0 \times 10^8)(10 \times 10^{-9})}{2} = 1.5 \text{ m} = 1500 \text{ mm}.$$

The VL53L1X performs this timing process internally and then gives the final distance measurement in millimetres through its digital register interface. In this project, the microcontroller first boots the sensor, clears the default interrupt, runs the initialization sequence, selects long-distance mode. After this, the program waits until the sensor shows that data is ready. Once it is ready, the measured distance is read in millimetres, and the interrupt is cleared so the next reading can be taken. This process repeats for every angle, which is what allows the scanner to move step by step until the full sweep is finished.

Communication between the microcontroller and the ToF sensor is done using I2C. PB2 is configured as SCL and PB3 is configured as SDA through I2C0. The sensor uses address 0x29. Since the TOF board already includes the voltage regulator and logic-level shifting circuitry, it can be connected directly to the 3.3 V MCU logic used in this design [5]. The system uses a fixed set of angular positions, so each transmitted sample is represented using the scan index, the angle in degrees, and the measured distance in mm.

The scan uses 128 step-based samples across a full 0° to 360° sweep. This gives an angular resolution of about 2.8125° per sample. This also shows the total number of measurements clearly. One scan has 128 distance readings. This means that a hallway scanned 5 times produces 640 total distance measurements, while a 3-slice demonstration scan produces 24 measurements. The theoretical quantization error of the distance reading is ±0.5 mm because the sensor output is given in whole millimetres [5]. This error is very small compared to the larger sources of error in the system, such as the reflectivity of the target, ambient lighting, the positioning of the stepper motor and the discrepancy in the manual movement during scanning.

Data Processing / Conversion

As it has come up countless times, it is important to note that the MCU itself does not convert raw distance measurements into 3D coordinates. The MCU's functionality is primarily to perform the acquisition it is the heart and brain of the system, forwarding instructions. In addition, it also performs validity checks and text transmission, while the PC that the AZ-2DX3_DELVIE-01 is plugged into performs the trigonometric conversion into Cartesian coordinates. MATLAB converts each valid scan into Cartesian coordinates using:

$$X = n\Delta x, \quad Y = dcos(\theta), \quad Z = dsin(\theta)$$

where n is the index number of the scan, Δx is the user-defined displacement between scans, d is the distance that is measured, and θ is the angle at which the distance was taken.

- x: displacement between scans,
- y: horizontal component of the scan,

- z: vertical component of the scan.

The angle itself comes from the step pattern, for this specific device, one full rotation is sampled vias 128 angular positions. As a result, the angle increment is by a factor of:

$$\Delta\theta = \frac{360}{128} = 2.8125^{\circ}$$

To determine the angle of sample number k:

$$\theta_k = k(2.8125^{\circ})$$

As an example, if k = 16, it corresponds to:

$$\theta_{16} = 16(2.8125^{\circ}) = 45^{\circ}$$

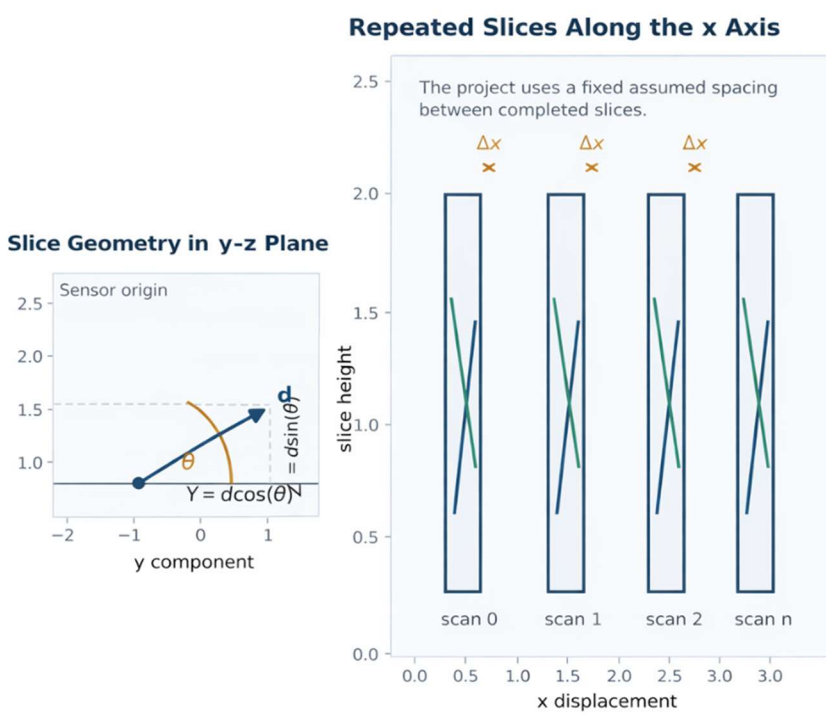

Figure 3: Measurement Geometry

To better understand the calculations performed, an example is shown below. If scan $n = 2$, displacement $\Delta x = 300$ mm, $d = 1500$mm and angle $\theta = 45^{\circ}$:

$$X = 2(300) = 600\ mm, \quad Y = 1500cos(45) = 1060.7\ mm, \quad Z = 1500sin(45) = 1060.7\ mm$$

These calculations are here just for reference; the PC carries out these calculations directly in MATLAB as it can perform these operations more reliably and such that the system does not become limited by computation and sacrifice measurement.

For invalid measurements, it is filtered in two stages. Within the MCU itself, measurements that are flagged by range status codes or impossible distances (greater than 4000 mm), are marked with sentinel code 65535. Within MATLAB, lines that have this marked value are rejected. For an extra layer of security, the MATLAB code still checks to see if any measurement indices are invalid beyond the marker.

Displacement Implementation

The DELVIE requires that fixed displacement occurs along the x-axis given the nature of the VL53L1X being a sensor capable of scanning only in two dimensions. For this device, displacement is implemented manually within the code. After one complete scan, the user must physically move the scanner to the next location and start the scan. **The software does not measure this displacement for the user, please be advised.** It assigns a fixed displacement value, and the user is required to move the device that same distance after each scan.

3.2 Visualization

Visualization is done within MATLAB using SERIAL I/O and 3D plotting libraries. The code opens the serial connection to the MCU, waits for the data, parses each line, rejects invalid samples, converts valid ones into 3D coordinates and writes the final 3D map cloud as an .xyz file for plotting.

The MATLAB functions used are:

- serialport and readline for UART communication,
- sscanf and strtrim for parsing.
- deg2rad, cos, and sin for coordinate conversion,
- writematrix for saving the point cloud as .xyz
- scatter3, plot3, sortrows, unique, axis equal, and view(3) for rendering the 3D model

Samples that are apart of the same scan are connected to form a closed loop, while samples that are taken across from each other and have the same angle are connected to show how the surface changes along the displacement.

The procedure is as defined:

1. MATLAB opens COM connection using serialport(port,115200).
2. Reads each UART frame with readline, removes whitespace using strtrim, parses commas using sscanf(line, %f,%f,%f), from which each UART frame is sent in the order: scan number, angle, distance.
3. Invalid data rejected based on checking if range status is 4 or 7 and if the distance exceeds 4000mm.
4. Final coordinate computed using trigonometric conversion as define previously.
5. Each coordinate is appended to the data matrix using the form. [scanIndex, angle, X,Y,Z,distance].

6. MATLAB render map using figure, hold on, grind on, axis equal and view(3) such that it is displayed in a 3D perspective and scaling along the axes.
7. xlabel, ylabel, zlabel and title are used to plot and organize the 3D Map into readable form.

4. Application Steps and Example

4.1 Setup and Use

The purpose of the section is to explain how to setup and use the device in a simple way. Note that it is NOT necessary to setup the development environment with any debugger since it is already done in the firmware.

The following are assumed:

- The MCU is loaded with the correct code and firmware.
- The correct version of MATLAB has been installed and the correct scanner.m file is available, open and running on the PC.
- The hardware for the scanner including the respective motor, driver board, scanner and MCU has been assembled in accordance with the device characteristics table defined in section 2.

The complete setup is:

- Connect the MSP432E401Y to the PC through the USB cable, establish both data and power connection. This uses the UART0 interface. Once plugged in, the PC creates a VCOM port that should be identified and used/mentioned in MATLAB for UART.
- To identify the VCOM port, open the Windows Device Manager right clicking the Windows Icon on the task bar and pressing device manager. Identify which COM port the MCU uses by opening the Ports tab in the Device Manager and find the port labelled with “XDS110”. This COM number MUST match the **port variable** within the scanner.m MATLAB code.
- Open scanner.m in MATLAB and confirm: baudrate = 115200, NUM_ANGLES = 128. you may set the following values to your direction: NUM_SCANS, SCAN_SPACING_MM. NUM_SCANS defines how many scans you want to take, and SCAN_SPACING_MM defines the displacement between each scan.
- Confirm the connections: the VL53L1X must be connected to PB2 and PB3 for I^2C. The stepper motor must be connected to pins PH0-PH3, and PJ0, although an onboard switch, must be unobstructed and available for use, ensure that no code currently loaded utilizes this button for any other functionality.
- Run scanner.m, The script should open the serial port, clear any remaining data and wait for the user to press enter. At this stage, MATLAB is ready to receive incoming data from the scanner.
- To begin data collection, press PJ0. The device now starts spinning through the entire rotation.
- Wait until one scan is complete. During scanning LED PN1 confirms if measurements are being taken, PN0 confirms if data is being sent to the PC, and PF4 turns on when one scan is complete.

- Move the scanner forward based on the SCAN_SPACING_MM value set. This movement creates the next x-axis scan in the 3D map. After moving, press PJ0 again to start the next scan. You do not have to run the MATLAB script again.
- Repeat the previous step until all scans have been captured, and then a 3D map should be made.

4.2 Expected Output

The expected result is a 3D point cloud using the convention as discussed in the project specifications, where the y-z plane represents each vertical distance slice, and the x represents the displacement between slices. For the capture, the output should show the general shape of the room, major openings and doorways, and the structure of the ceiling/floor enough that it is recognizable what is being scanned. More specifically, for this example, the output should have:

- the y-z plane representing one slice across a full sweep
- x-axis representing the 300mm displacement between these slices
- continuity should be shown and the depth should easily be analysed.

A successful scan should not appear as random points, but instead recognizable geometry should be highlighted such as long wall surfaces, corners, openings and the ceiling/floor. Irregularities are expected because of the requirement of manual displacement, but the overall shape should be identifiable. Consider the following scan result:

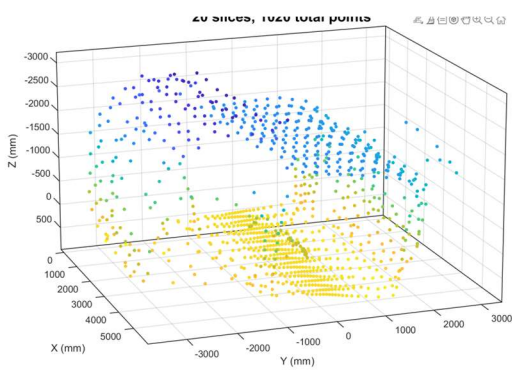

Figure 4: 3D Point Cloud

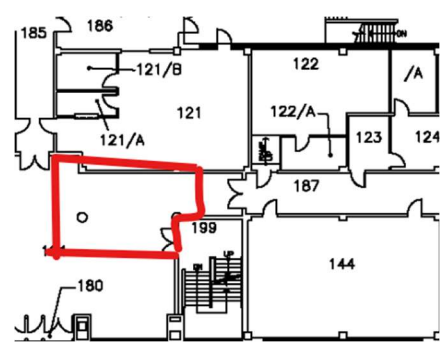

Figure 6: Location

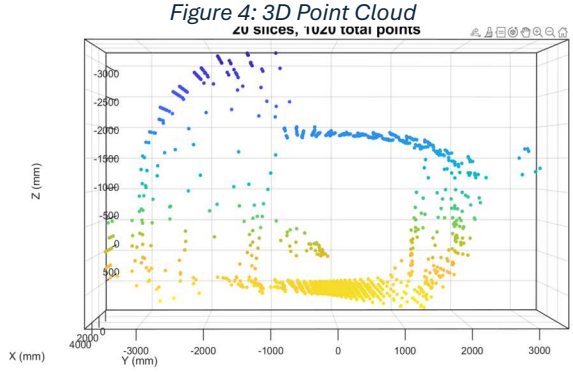

Figure 5: Point Cloud Front View

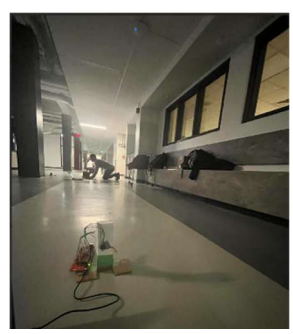

Figure 7: Real Life Location Image

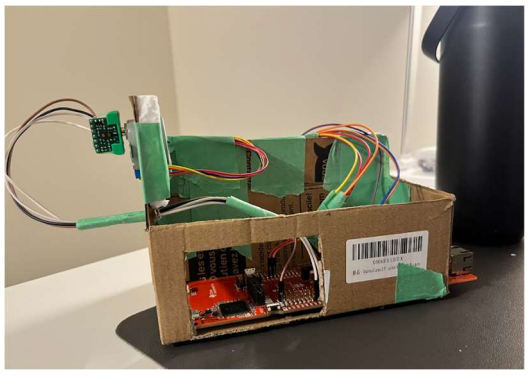

Figure 8: Physical Prototype

5. Limitations

1. Microcontroller floating-point and trigonometric limitations

The MCU uses an ARM Cortex-M4F core with a single-precision floating-point unit, so it does support floating-point operations, but it is still more limited than doing the same calculations on the PC [2]. In this project, the main issue is not that trigonometric functions cannot be done, but that doing them repeatedly for every scan angle would increase execution time and also make the code more complicated for no real benefit to the acquisition itself. For this reason, the MCU only sends the scan number, angle, and distance, while the sin and cos calculations are done later in MATLAB. This shows directly why the floating-point limitations of the microcontroller matter in the final design when trigonometric functions are involved.

2. Maximum serial rate with PC

The maximum standard UART communication rate used in this project is 115200 bps, and this is also the rate used on the UART0 connection to the PC. The UART driver sets the divisor registers for this baud rate, and this can be verified by checking that the transmitted data is received correctly on the PC and that the TX bit period is about 1/115200, which is about 8.68 µs, on the oscilloscope or logic analyzer. Since both the MCU UART configuration and the MATLAB receiver are set to 115200 bps, the serial communication link is matched from end to end.

3. Communication Method and speed between MCU and TOF

The communication between the ToF sensor and the microcontroller is done using I2C. In the final code, I2C0 MTPR is set to 0x0002803B, which gives a timer period register value of 59. Using a 36 MHz system clock, the resulting I2C clock speed becomes:

$$f_{\mathrm{SCL}} = \frac{36 \times 10^6}{2(6+4)(59+1)} \approx 30 \text{ kbit/s}.$$

4. Primary Speed Bottlenecks

The main speed bottleneck of the full system is the ToF measurement timing, not the UART transmission. For each angular position, the sensor uses a 100 ms timing budget and a 120 ms inter-measurement period. This means the measurement process itself takes up most of the scan time. In the hallway version, one full scan uses 128 angular positions. As a result, the measurement stage alone takes about 128 × 120 ms = 15.36 s, even before including the stepper motor movement, UART transmission, and the manual repositioning between scans. This shows that the stepper motor is only a secondary bottleneck in the system. The largest user-added bottleneck is the manual repositioning between slices. UART is also not the primary bottleneck because each transmitted line is short, and the 115200 bps serial link can send the text data faster than the sensor can produce new measurements. This can be verified by comparing the expected sensor timing to the observed full scan time, and by noting that reducing the UART text does not change the total scan time as much as reducing the number of measurements or reducing the time needed to manually move the device. For this reason, the main speed bottleneck in the final system is the acquisition path itself, not the communication path.

6. Circuit Schematic

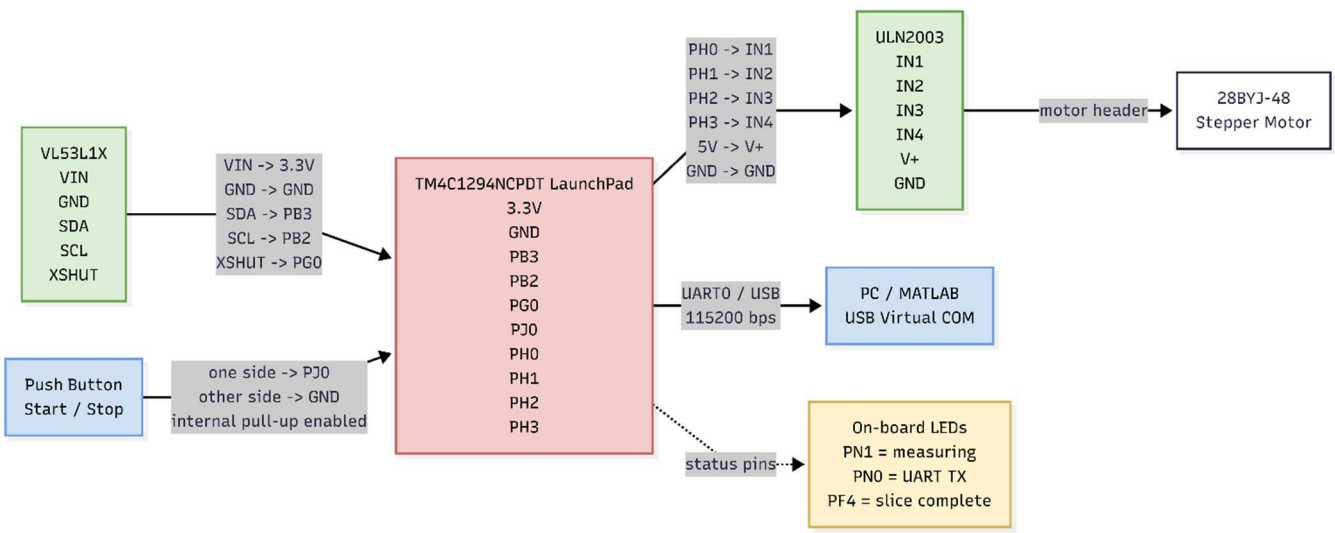

Figure 9: Circuit Schematic

7. Programming Logic Flowchart

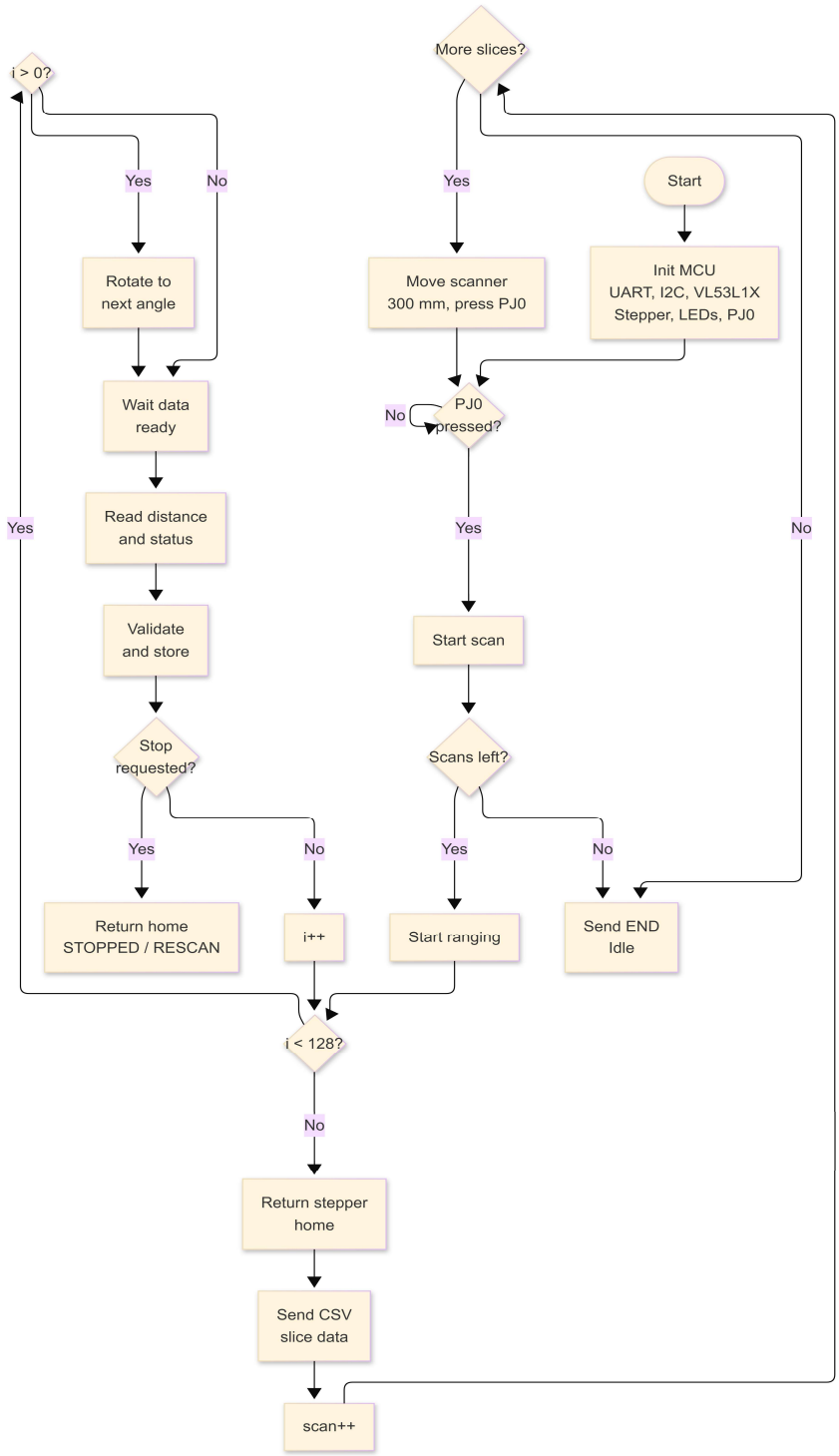

Figure 10: Logic Programming Flowchart

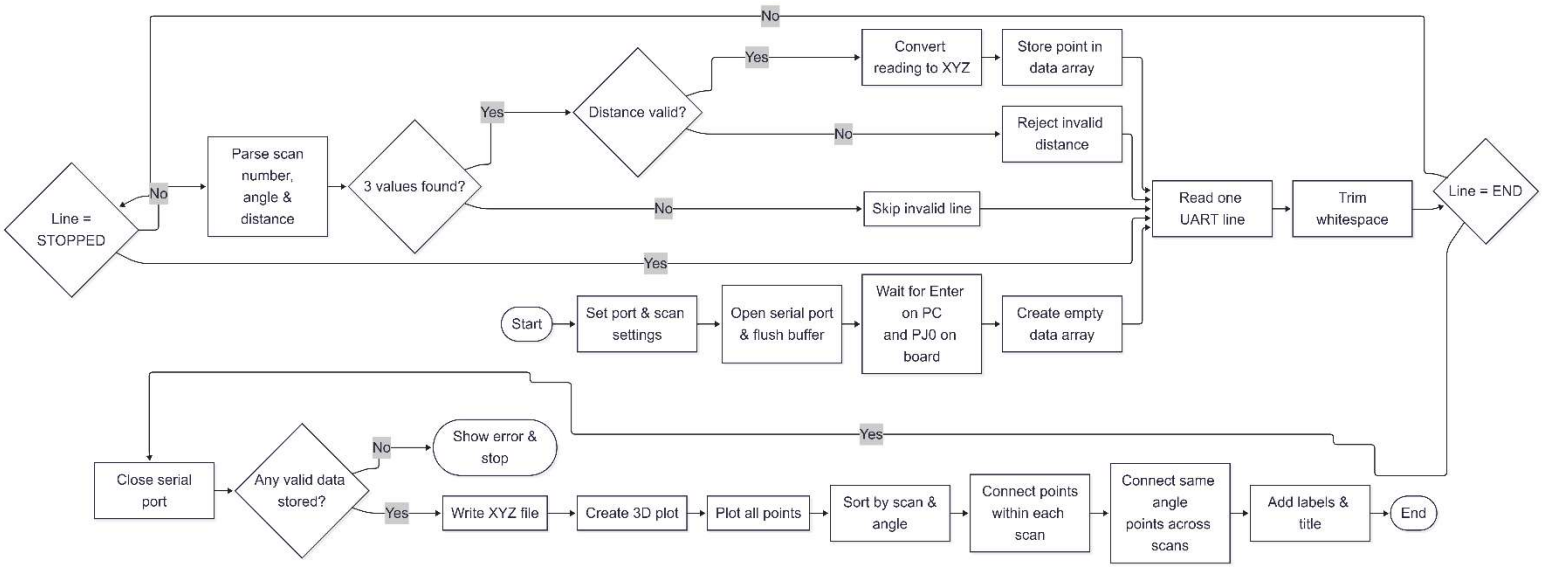

Figure 11: PC Code FlowChart

References

[1] McMaster University, “2025–2026 2DX3 Project Specification – Observe, Reason, Act: Spatial Mapping Using Time-of- Flight,” Winter 2026.

[2] Texas Instruments, “MSP432E401Y Microcontroller Data Sheet,” Rev. B. [Online]. Available: https://www.ti.com/lit/ds/symlink/msp432e401y.pdf?ts=1775590656422&ref_url=https%253A%252F%252Fwww.google.com%252F

[3] Texas Instruments, “MSP432E401Y User’s Guide,” Rev. C. [Online]. Available: https://www.ti.com/lit/ug/spmu365c/spmu365c.pdf

[4] STMicroelectronics, “VL53L1X Datasheet: A New Generation, Long Distance Ranging Time-of-Flight Sensor,” [Online]. Available: https://www.st.com/resource/en/datasheet/vl53l1x.pdf

[5] Pololu Corporation, “VL53L1X Time-of-Flight Distance Sensor Carrier with Voltage Regulator, 400 cm Max,” [Online]. Available: https://www.pololu.com/product-info-merged/3415